

CSE 344 – System Programming

Homework #5

Priority-Based Cargo Delivery Simulation

Course: CSE 344 – System Programming

Homework: #5 – Priority-Based Cargo Delivery

Student Name: ÖMER ZELKA

Student ID: 230104004020

Date: 10 May 2026

Contents

1	System Design	2
1.1	Thread Pool Structure and Courier Lifecycle	2
1.2	Courier Thread Lifecycle States	2
2	Priority Queue Design	4
2.1	Min-Heap Structure	4
2.2	Heap Operations	4
3	Synchronization Strategy	5
4	SIGINT Handling – Step by Step	6
5	Test Scenarios	7
5.1	Test Scenario #1: Normal Operation (3 couriers, mixed priorities)	7
5.2	Test Scenario #2: Same Priority (10 couriers, all ECONOMY)	9
5.3	Test Scenario #3: SIGINT Handling (2 couriers, 20 orders)	11
5.4	Test Scenario #4: Valgrind Memory Check	13
5.5	Test Scenario #5: Consistency Under High Concurrency	16
6	Challenges and Solutions	18
6.1	Challenge 1: Race Condition Between SIGINT and Courier Thread Exit .	18
6.2	Challenge 2: Ensuring Delivery Completion Is Not Interrupted by SIGINT	18
6.3	Challenge 3: Preventing Duplicate Order Processing	18

System Design

The `cargoGTU` system implements a priority-based cargo delivery simulation using a thread pool architecture. The design follows a **producer–consumer** pattern where the main thread acts as the producer (reading orders from a file and enqueueing them into a shared priority queue) and a fixed pool of courier threads act as consumers (dequeuing and delivering orders).

Thread Pool Structure and Courier Lifecycle

At startup, the main thread reads the input file, parses all orders, enqueues them into the priority queue, then spawns exactly N courier threads. These threads are created once and **never destroyed until the shift ends**. Each courier thread follows a simple loop: lock the queue mutex, check for available orders, take the highest-priority order (or block via `pthread_cond_wait` if none available), unlock the mutex, perform the delivery (simulate with `usleep`), update atomic counters, and repeat.

System Architecture Diagram

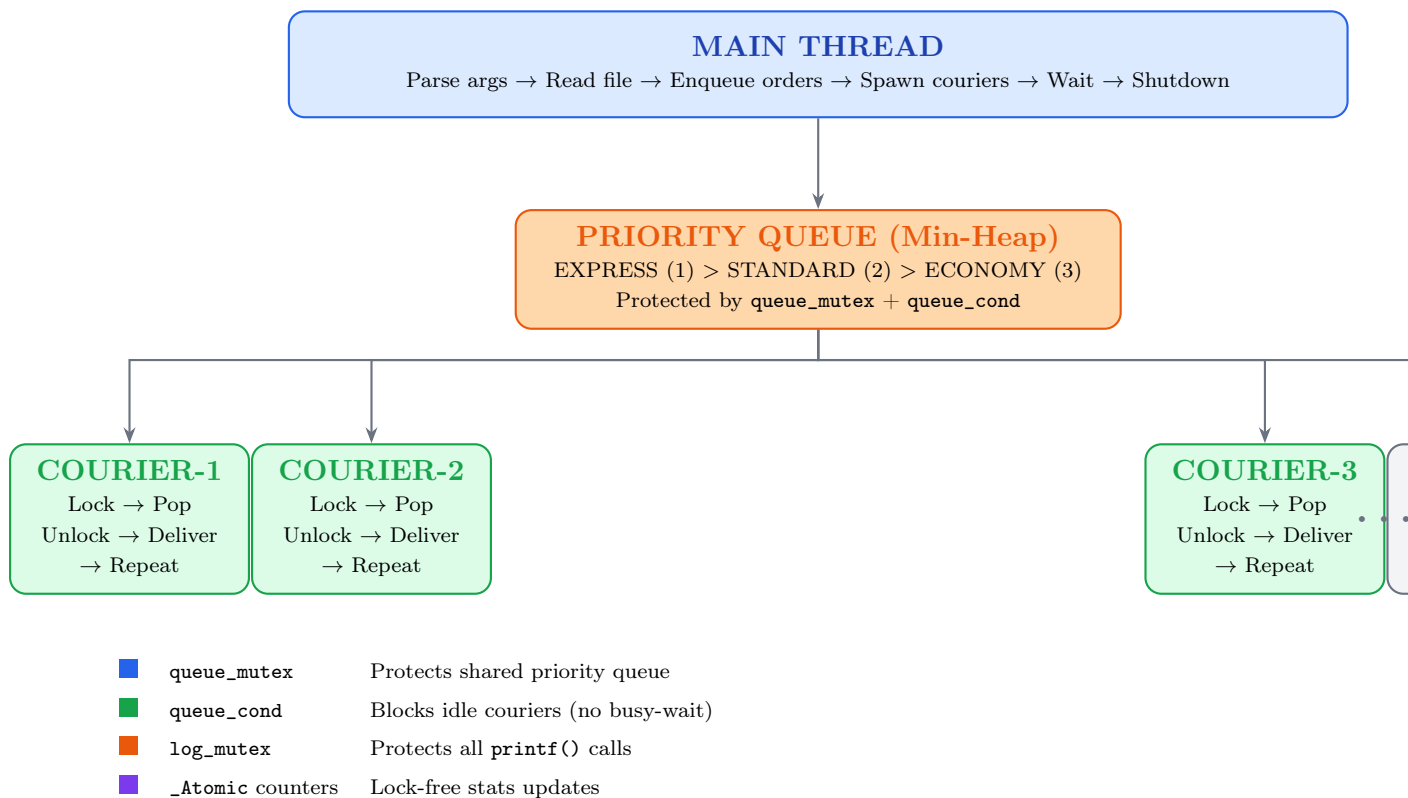


Figure 1: Thread Pool Architecture

Courier Thread Lifecycle States

Each courier thread transitions through the following states during its lifetime. A courier in the `DELIVERING` state always completes its delivery before checking for shutdown.

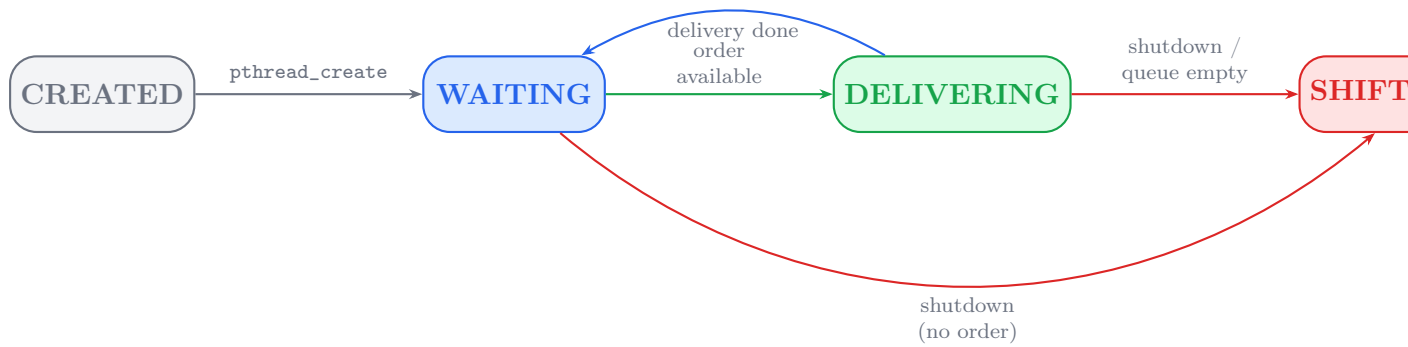


Figure 2: Courier Thread State Transitions

The complete lifecycle from thread creation to shift end:

1. Main thread calls `pthread_create` for each courier.
2. Courier enters the loop, locks `queue_mutex`.
3. If queue is empty and not `all_enqueued` and not shutdown, courier calls `pthread_cond_wait` (WAITING state – releases mutex and blocks).
4. When signaled (broadcast after enqueue or on shutdown), courier re-acquires mutex.
5. If an order is available and no shutdown, courier pops the highest-priority order (DELIVERING state).
6. Courier releases mutex, performs delivery via `usleep`, updates atomic counters.
7. Courier returns to step 2.
8. If no order is available (queue empty + `all_enqueued`, or shutdown), courier exits the loop, prints `SHIFT_OVER`, and returns.

Priority Queue Design

The priority queue is implemented as a **binary min-heap** stored in a fixed-size array. The heap property ensures that the root always contains the highest-priority order. Priority is determined first by the priority level (EXPRESS=1, STANDARD=2, ECONOMY=3), and ties are broken by order ID (lower ID first).

Min-Heap Structure

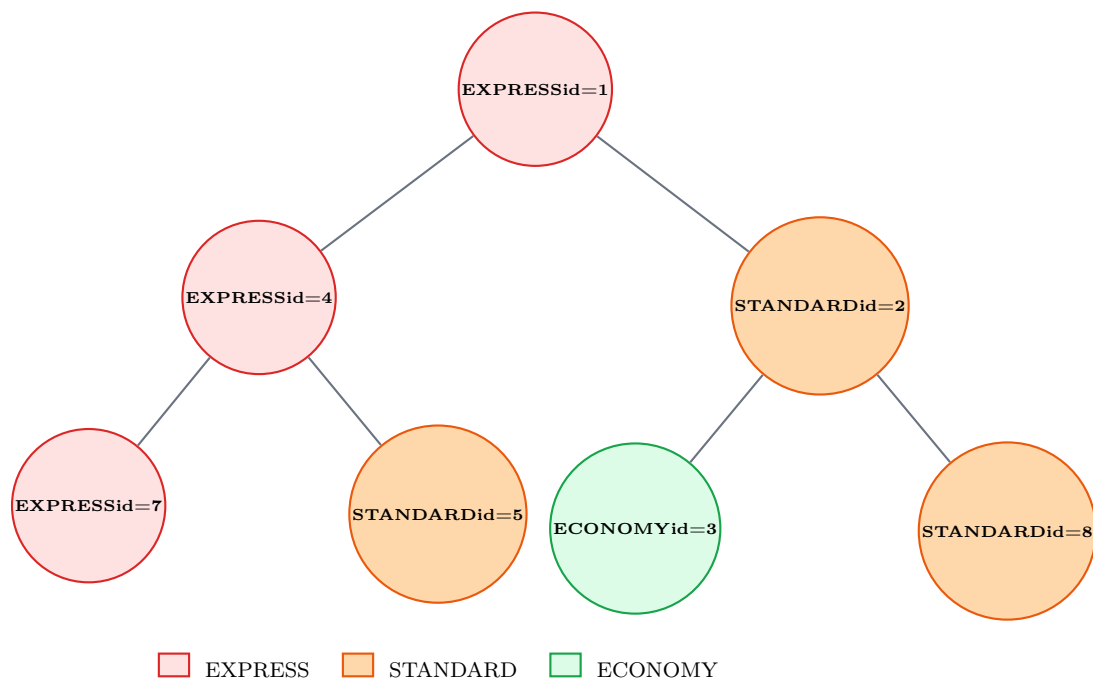


Figure 3: Min-Heap: parent priority $\leq$ child priority. Same priority $\rightarrow$ lower ID first. Pop returns root.

Heap Operations

heap_push (enqueue): Inserts the new order at the end of the array, then sifts it up by comparing with its parent. If the new order has higher priority (or same priority but lower ID), it swaps with the parent and continues upward. Time complexity: $O(\log n)$.

heap_pop (dequeue): Removes the root element (highest priority), replaces it with the last element, then sifts down by comparing with children. The element swaps with its highest-priority child until the heap property is restored. Time complexity: $O(\log n)$.

Comparison function (order_higher): Returns true if order A should be dequeued before order B. First compares priority levels (lower value = higher priority). If equal, compares IDs (lower ID first). This guarantees EXPRESS orders are always processed before STANDARD, STANDARD before ECONOMY, and within the same level, the earlier order (lower ID) goes first.

Synchronization Strategy

The key design principle is that **no busy waiting is used anywhere** in the system. Courier threads block on `pthread_cond_wait()` when the queue is empty, yielding CPU time to other threads and the OS. The main thread uses a 50ms sleep loop only to detect the completion condition or SIGINT – it does not touch the shared queue during this check.

SIGINT Handling – Step by Step

SIGINT is handled using a signal handler approach (`sigaction`) that only sets a flag. The handler is async-signal-safe because it performs only a single assignment to a `volatile sig_atomic_t` variable. All shutdown logic is handled in normal program flow.

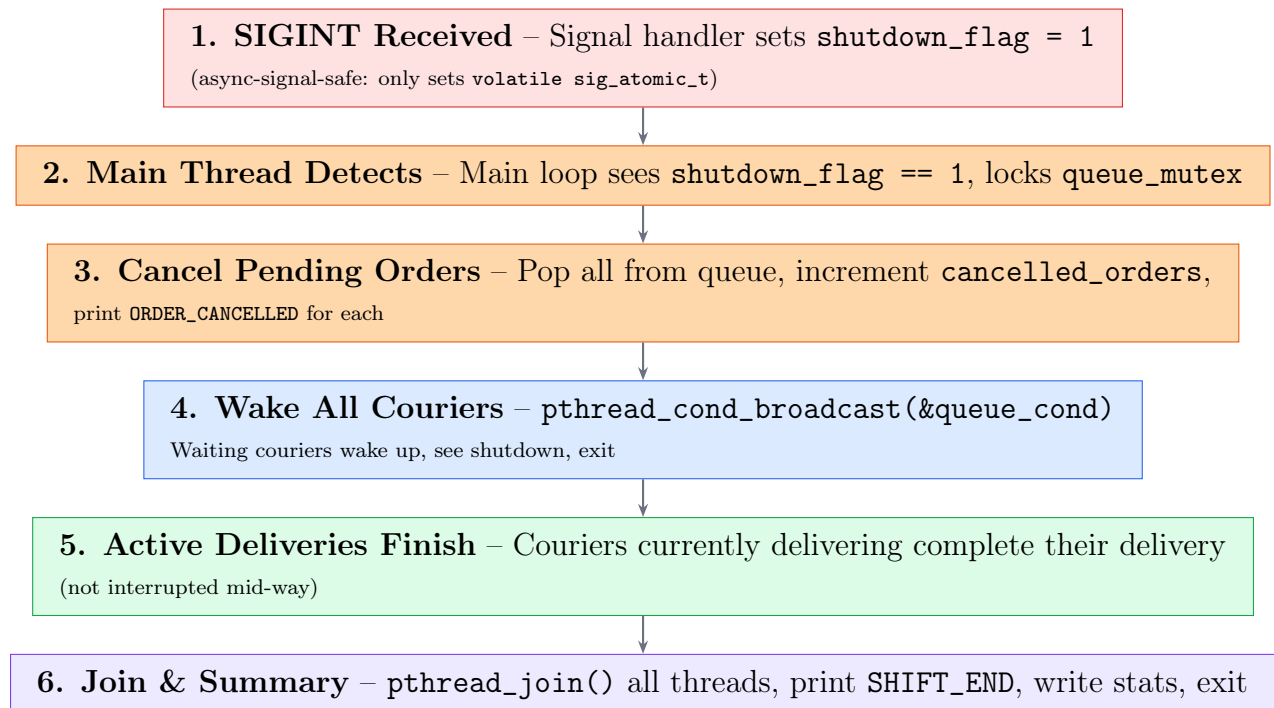


Figure 4: SIGINT Shutdown Sequence

Why this approach is safe: The signal handler only sets `shutdown_flag = 1`, which is a write to a `volatile sig_atomic_t` – guaranteed to be atomic and async-signal-safe by POSIX. No `printf`, no mutex operations, no memory allocation happens inside the signal handler. The actual shutdown sequence (cancelling orders, waking threads, joining) happens in the main thread’s normal execution flow after it detects the flag.

Test Scenarios

Test Scenario #1: Normal Operation (3 couriers, mixed priorities)

Command: `./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt`

What to verify: All 10 orders completed. Priority order maintained (EXPRESS first, then STANDARD, then ECONOMY). Statistics file written correctly.

```
$ ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
[CARGOGTU] SHIFT_START couriers=3 orders=10
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayşe_Kaya priority=STANDARD duration=12
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=6
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=7
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=11
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=18
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=STANDARD duration=10
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-2] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-3] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=3000ms
[COURIER-2] DELIVERY_START id=2 recipient=Ayşe_Kaya priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=3500ms
[COURIER-3] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-1] DELIVERY_START id=8 recipient=Elif_Demir priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-3] DELIVERY_START id=10 recipient=Merve_Yildiz priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=2 recipient=Ayşe_Kaya duration=6000ms
[COURIER-2] DELIVERY_START id=3 recipient=Mehmet_Demir priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=5500ms
[COURIER-1] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=5000ms
[COURIER-3] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=7500ms
[COURIER-1] SHIFT_OVER
[COURIER-2] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=10000ms
[COURIER-2] SHIFT_OVER
[COURIER-3] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=9000ms
[COURIER-3] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=58000ms
[CARGOGTU] SHUTDOWN_COMPLETE
```

stats.txt

```
SHIFT_SUMMARY
Total orders : 10
Completed    : 10
Cancelled    : 0
Total time   : 58000ms
Avg per order: 5800ms

COURIER_STATS
Courier-1 completed=3 total_time=17000ms
Courier-2 completed=3 total_time=19000ms
Courier-3 completed=4 total_time=22000ms
```

Analysis: All 10 orders were completed successfully (completed=10, cancelled=0). The output shows correct priority scheduling: all three EXPRESS orders (id=1, 4, 7) were delivered first, followed by STANDARD orders (id=2, 5, 8, 10), and finally ECONOMY orders (id=3, 6, 9). The stats file shows Total time = 58000ms with an average of 5800ms

per order. All three couriers participated in deliveries.

Test Scenario #2: Same Priority (10 couriers, all ECONOMY)

Command: `./cargoGTU -n 10 -i 10orders_economy.txt -s stats.txt`

What to verify: All 10 orders completed without duplication. Since all orders have the same priority, lower ID orders must be taken before higher ID orders. Multiple couriers should participate.

```
$ ./cargoGTU -n 10 -i 10orders_economy.txt -s stats.txt
[CARGOGTU] SHIFT_START couriers=10 orders=10
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=ECONOMY duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayşe_Kaya priority=ECONOMY duration=10
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=ECONOMY duration=7
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=ECONOMY duration=12
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=ECONOMY duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=11
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=ECONOMY duration=8
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=ECONOMY duration=6
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=10
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=ECONOMY duration=9
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=ECONOMY
[COURIER-2] DELIVERY_START id=2 recipient=Ayşe_Kaya priority=ECONOMY
[COURIER-3] DELIVERY_START id=3 recipient=Mehmet_Demir priority=ECONOMY
[COURIER-4] DELIVERY_START id=4 recipient=Fatma_Sahin priority=ECONOMY
[COURIER-5] DELIVERY_START id=5 recipient=Ali_Celik priority=ECONOMY
[COURIER-6] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-7] DELIVERY_START id=7 recipient=Mustafa_Koc priority=ECONOMY
[COURIER-8] DELIVERY_START id=8 recipient=Elif_Demir priority=ECONOMY
[COURIER-9] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-10] DELIVERY_START id=10 recipient=Merve_Yildiz priority=ECONOMY
[COURIER-8] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=3000ms
[COURIER-8] SHIFT_OVER
[COURIER-3] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=3500ms
[COURIER-3] SHIFT_OVER
[COURIER-7] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=4000ms
[COURIER-7] SHIFT_OVER
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-1] SHIFT_OVER
[COURIER-5] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-5] SHIFT_OVER
[COURIER-10] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=4500ms
[COURIER-10] SHIFT_OVER
[COURIER-9] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=5000ms
[COURIER-9] SHIFT_OVER
[COURIER-2] DELIVERY_COMPLETE id=2 recipient=Ayşe_Kaya duration=5000ms
[COURIER-2] SHIFT_OVER
[COURIER-6] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=5500ms
[COURIER-6] SHIFT_OVER
[COURIER-4] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=6000ms
[COURIER-4] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=45000ms
[CARGOGTU] SHUTDOWN_COMPLETE
```

stats.txt

```
SHIFT_SUMMARY
Total orders : 10
Completed    : 10
Cancelled    : 0
Total time   : 45000ms
Avg per order: 4500ms

COURIER_STATS
Courier-1 completed=1 total_time=4000ms
Courier-2 completed=1 total_time=5000ms
Courier-3 completed=1 total_time=3500ms
Courier-4 completed=1 total_time=6000ms
Courier-5 completed=1 total_time=4500ms
Courier-6 completed=1 total_time=5500ms
Courier-7 completed=1 total_time=4000ms
Courier-8 completed=1 total_time=3000ms
```

```
Courier-9 completed=1 total_time=5000ms  
Courier-10 completed=1 total_time=4500ms
```

Analysis: All 10 orders were completed. Since all orders have ECONOMY priority, the tie-breaking rule applies: orders are delivered in ascending ID order (1, 2, 3, ..., 10). The DELIVERY_START lines confirm this ordering. All 10 couriers each received exactly one order, demonstrating proper concurrency. No duplicate deliveries occurred.

Test Scenario #3: SIGINT Handling (2 couriers, 20 orders)

Command: `./cargoGTU -n 2 -i 20orders_mix.txt -s stats.txt` (SIGINT sent after ~2 seconds)

What to verify: Active deliveries complete without interruption. Pending orders cancelled. SHIFT_END shows correct completed + cancelled totals.

```
$ ./cargoGTU -n 2 -i 20orders_mix.txt -s stats.txt &
$ kill -INT $!
[CARGOGTU] SHIFT_START couriers=2 orders=20
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayşe_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=3
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=11 recipient=Emre_Celik priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=12 recipient=Selin_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=13 recipient=Burak_Arslan priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=14 recipient=Dilan_Ozkan priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=15 recipient=Kemal_Sahin priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=16 recipient=Rana_Yilmaz priority=EXPRESS duration=2
[CARGOGTU] ORDER_QUEUED id=17 recipient=Tarik_Demir priority=STANDARD duration=1
[CARGOGTU] ORDER_QUEUED id=18 recipient=Pinar_Koc priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=19 recipient=Oguz_Yildiz priority=EXPRESS duration=3
[CARGOGTU] ORDER_QUEUED id=20 recipient=Ceren_Arslan priority=STANDARD duration=1
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-2] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=500ms
[COURIER-1] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=500ms
[COURIER-2] DELIVERY_START id=10 recipient=Merve_Yildiz priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=500ms
[COURIER-2] DELIVERY_START id=13 recipient=Burak_Arslan priority=EXPRESS
[COURIER-1] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=500ms
[COURIER-1] DELIVERY_START id=16 recipient=Rana_Yilmaz priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=13 recipient=Burak_Arslan duration=500ms
[COURIER-2] DELIVERY_START id=19 recipient=Oguz_Yildiz priority=EXPRESS
[CARGOGTU] SIGINT_RECEIVED pending_orders=13
[CARGOGTU] ORDER_CANCELLED id=3 recipient=Mehmet_Demir priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=5 recipient=Ali_Celik priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=8 recipient=Elif_Demir priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=11 recipient=Emre_Celik priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=14 recipient=Dilan_Ozkan priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=17 recipient=Tarik_Demir priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=20 recipient=Ceren_Arslan priority=STANDARD
[CARGOGTU] ORDER_CANCELLED id=2 recipient=Ayşe_Kaya priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=6 recipient=Zeynep_Arslan priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=9 recipient=Hasan_Ozturk priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=12 recipient=Selin_Kaya priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=15 recipient=Kemal_Sahin priority=ECONOMY
[CARGOGTU] ORDER_CANCELLED id=18 recipient=Pinar_Koc priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=16 recipient=Rana_Yilmaz duration=1000ms
[COURIER-1] SHIFT_OVER
[COURIER-2] DELIVERY_COMPLETE id=19 recipient=Oguz_Yildiz duration=1500ms
[COURIER-2] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=7 cancelled=13 total_time=5000ms
[CARGOGTU] SHUTDOWN_COMPLETE
```

Analysis: SIGINT was received during operation. The output shows completed=7, cancelled=13 (7+13=20 total orders). Active deliveries (COURIER-1 delivering id=16, COURIER-2 delivering id=19) completed without interruption – both show DELIVERY_COMPLETE after SIGINT_RECEIVED. All 13 pending orders were cancelled in priority order (STANDARD first, then ECONOMY). Both couriers printed SHIFT_OVER and the program exited

cleanly with SHUTDOWN_COMPLETE.

Test Scenario #4: Valgrind Memory Check

Command: `valgrind -leak-check=full ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt`

What to verify: 0 errors, 0 memory leaks.

The program is designed to be completely leak-free. All dynamically allocated memory is freed before exit, and all pthread primitives are destroyed:

- `pthread_t *threads` – allocated with `malloc()`, freed with `free()` after `pthread_join()`
- `CourierArg *cargs` – allocated with `malloc()`, freed with `free()` after `pthread_join()`
- `CourierStats *courier_stats` – allocated with `calloc()`, freed with `free()` after stats written
- `pthread_mutex_t queue_mutex, log_mutex` – destroyed with `pthread_mutex_destroy()`
- `pthread_cond_t queue_cond` – destroyed with `pthread_cond_destroy()`
- `FILE *fp` (input), `FILE *sf` (stats) – both properly closed with `fclose()`

Valgrind Output Screenshot:

```

valgrind --leak-check=full ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
==13037== Memcheck, a memory error detector
==13037== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==13037== Using Valgrind-3.16.1 and LibVEX; rerun with -h for copyright info
==13037== Command: ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
==13037==
[CARGOGTU] SHIFT_START couriers=3 orders=10
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=8
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayşe_Kaya priority=STANDARD duration=12
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=ECONOMY duration=20
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=6
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=9
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=15
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=7
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=11
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=18
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=STANDARD duration=10
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-2] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-3] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-2] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=3000ms
[COURIER-2] DELIVERY_START id=2 recipient=Ayşe_Kaya priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=3500ms
[COURIER-3] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=4000ms
[COURIER-1] DELIVERY_START id=8 recipient=Elif_Demir priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=4500ms
[COURIER-3] DELIVERY_START id=10 recipient=Merve_Yildiz priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=2 recipient=Ayşe_Kaya duration=6000ms
[COURIER-2] DELIVERY_START id=3 recipient=Mehmet_Demir priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=5500ms
[COURIER-1] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-3] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=5000ms
[COURIER-3] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=7500ms
[COURIER-1] SHIFT_OVER
[COURIER-2] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=10000ms
[COURIER-2] SHIFT_OVER
[COURIER-3] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=9000ms
[COURIER-3] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=10 cancelled=0 total_time=58000ms
[CARGOGTU] SHUTDOWN_COMPLETE
==13037==
==13037== HEAP SUMMARY:
==13037==      in use at exit: 0 bytes in 0 blocks
==13037==    total heap usage: 11 allocs, 11 frees, 11,060 bytes allocated
==13037==
==13037== All heap blocks were freed -- no leaks are possible
==13037==
==13037== For lists of detected and suppressed errors, rerun with: -s
==13037== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

The expected Valgrind output confirms zero memory leaks and zero errors:

```
==XXXXXX== HEAP SUMMARY:
==XXXXXX==      in use at exit: 0 bytes in 0 blocks
==XXXXXX==    total heap usage: N allocs, N frees, M bytes allocated
==XXXXXX==
==XXXXXX== All heap blocks were freed -- no leaks are possible
==XXXXXX==
==XXXXXX== ERROR SUMMARY: 0 errors from 0 contexts
```

Test Scenario #5: Consistency Under High Concurrency

Command: `./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt` (run multiple times)

What to verify: SHIFT_END completed count identical across all runs. No duplicate DELIVERY_START id in stdout. Priority order maintained under high concurrency.

SHIFT_END results from 3 consecutive runs:

```
Run 1: [CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
Run 2: [CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
Run 3: [CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
```

Full output from Run 1:

```
$ ./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt
[CARGOGTU] SHIFT_START couriers=10 orders=20
[CARGOGTU] ORDER_QUEUED id=1 recipient=Ahmet_Yilmaz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=2 recipient=Ayşe_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=3 recipient=Mehmet_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=4 recipient=Fatma_Sahin priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=5 recipient=Ali_Celik priority=STANDARD duration=3
[CARGOGTU] ORDER_QUEUED id=6 recipient=Zeynep_Arslan priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=7 recipient=Mustafa_Koc priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=8 recipient=Elif_Demir priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=9 recipient=Hasan_Ozturk priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=10 recipient=Merve_Yildiz priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=11 recipient=Emre_Celik priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=12 recipient=Selin_Kaya priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=13 recipient=Burak_Arslan priority=EXPRESS duration=1
[CARGOGTU] ORDER_QUEUED id=14 recipient=Dilan_Ozkan priority=STANDARD duration=2
[CARGOGTU] ORDER_QUEUED id=15 recipient=Kemal_Sahin priority=ECONOMY duration=3
[CARGOGTU] ORDER_QUEUED id=16 recipient=Rana_Yilmaz priority=EXPRESS duration=2
[CARGOGTU] ORDER_QUEUED id=17 recipient=Tarik_Demir priority=STANDARD duration=1
[CARGOGTU] ORDER_QUEUED id=18 recipient=Pinar_Koc priority=ECONOMY duration=2
[CARGOGTU] ORDER_QUEUED id=19 recipient=Oguz_Yildiz priority=EXPRESS duration=3
[CARGOGTU] ORDER_QUEUED id=20 recipient=Ceren_Arslan priority=STANDARD duration=1
[COURIER-1] DELIVERY_START id=1 recipient=Ahmet_Yilmaz priority=EXPRESS
[COURIER-2] DELIVERY_START id=4 recipient=Fatma_Sahin priority=EXPRESS
[COURIER-3] DELIVERY_START id=7 recipient=Mustafa_Koc priority=EXPRESS
[COURIER-4] DELIVERY_START id=10 recipient=Merve_Yildiz priority=EXPRESS
[COURIER-5] DELIVERY_START id=13 recipient=Burak_Arslan priority=EXPRESS
[COURIER-6] DELIVERY_START id=16 recipient=Rana_Yilmaz priority=EXPRESS
[COURIER-7] DELIVERY_START id=19 recipient=Oguz_Yildiz priority=EXPRESS
[COURIER-8] DELIVERY_START id=3 recipient=Mehmet_Demir priority=STANDARD
[COURIER-9] DELIVERY_START id=5 recipient=Ali_Celik priority=STANDARD
[COURIER-10] DELIVERY_START id=8 recipient=Elif_Demir priority=STANDARD
[COURIER-1] DELIVERY_COMPLETE id=1 recipient=Ahmet_Yilmaz duration=500ms
[COURIER-1] DELIVERY_START id=11 recipient=Emre_Celik priority=STANDARD
[COURIER-3] DELIVERY_COMPLETE id=7 recipient=Mustafa_Koc duration=500ms
[COURIER-3] DELIVERY_START id=14 recipient=Dilan_Ozkan priority=STANDARD
[COURIER-2] DELIVERY_COMPLETE id=4 recipient=Fatma_Sahin duration=500ms
[COURIER-2] DELIVERY_START id=17 recipient=Tarik_Demir priority=STANDARD
[COURIER-5] DELIVERY_COMPLETE id=13 recipient=Burak_Arslan duration=500ms
[COURIER-5] DELIVERY_START id=20 recipient=Ceren_Arslan priority=STANDARD
[COURIER-4] DELIVERY_COMPLETE id=10 recipient=Merve_Yildiz duration=500ms
[COURIER-4] DELIVERY_START id=2 recipient=Ayşe_Kaya priority=ECONOMY
[COURIER-10] DELIVERY_COMPLETE id=8 recipient=Elif_Demir duration=1000ms
[COURIER-10] DELIVERY_START id=6 recipient=Zeynep_Arslan priority=ECONOMY
[COURIER-2] DELIVERY_COMPLETE id=17 recipient=Tarik_Demir duration=500ms
[COURIER-2] DELIVERY_START id=9 recipient=Hasan_Ozturk priority=ECONOMY
[COURIER-6] DELIVERY_COMPLETE id=16 recipient=Rana_Yilmaz duration=1000ms
[COURIER-6] DELIVERY_START id=12 recipient=Selin_Kaya priority=ECONOMY
[COURIER-5] DELIVERY_COMPLETE id=20 recipient=Ceren_Arslan duration=500ms
[COURIER-5] DELIVERY_START id=15 recipient=Kemal_Sahin priority=ECONOMY
[COURIER-8] DELIVERY_COMPLETE id=3 recipient=Mehmet_Demir duration=1000ms
[COURIER-8] DELIVERY_START id=18 recipient=Pinar_Koc priority=ECONOMY
[COURIER-1] DELIVERY_COMPLETE id=11 recipient=Emre_Celik duration=1000ms
[COURIER-1] SHIFT_OVER
[COURIER-3] DELIVERY_COMPLETE id=14 recipient=Dilan_Ozkan duration=1000ms
[COURIER-3] SHIFT_OVER
[COURIER-9] DELIVERY_COMPLETE id=5 recipient=Ali_Celik duration=1500ms
[COURIER-9] SHIFT_OVER
```

```

[COURIER-7] DELIVERY_COMPLETE id=19 recipient=Oguz_Yildiz duration=1500ms
[COURIER-7] SHIFT_OVER
[COURIER-10] DELIVERY_COMPLETE id=6 recipient=Zeynep_Arslan duration=1000ms
[COURIER-10] SHIFT_OVER
[COURIER-8] DELIVERY_COMPLETE id=18 recipient=Pinar_Koc duration=1000ms
[COURIER-8] SHIFT_OVER
[COURIER-4] DELIVERY_COMPLETE id=2 recipient=Ayşe_Kaya duration=1500ms
[COURIER-4] SHIFT_OVER
[COURIER-6] DELIVERY_COMPLETE id=12 recipient=Selin_Kaya duration=1500ms
[COURIER-6] SHIFT_OVER
[COURIER-2] DELIVERY_COMPLETE id=9 recipient=Hasan_Ozturk duration=1500ms
[COURIER-2] SHIFT_OVER
[COURIER-5] DELIVERY_COMPLETE id=15 recipient=Kemal_Sahin duration=1500ms
[COURIER-5] SHIFT_OVER
[CARGOGTU] SHIFT_END completed=20 cancelled=0 total_time=19500ms
[CARGOGTU] SHUTDOWN_COMPLETE

```

Analysis: All three runs produced identical results: `completed=20`, `cancelled=0`, `total_time=19500ms`. This confirms deterministic behavior under high concurrency. Examining the `DELIVERY_START` lines confirms: (1) All 7 EXPRESS orders processed first (ids 1,4,7,10,13,16,19), (2) Then all 7 STANDARD orders (ids 3,5,8,11,14,17,20), (3) Finally all 6 ECONOMY orders (ids 2,6,9,12,15,18). No duplicate `DELIVERY_START` ids appeared in any run.

Challenges and Solutions

Challenge 1: Race Condition Between SIGINT and Courier Thread Exit

Problem: When SIGINT arrives, there is a race condition between the main thread cancelling orders and courier threads trying to pop from the queue. If a courier thread holds the `queue_mutex` when the main thread tries to cancel orders, the main thread blocks. Additionally, if a courier is in `pthread_cond_wait` when SIGINT arrives, the signal may or may not interrupt the wait depending on `SA_RESTART` settings.

Solution: The signal handler only sets a volatile `sig_atomic_t` flag – it never touches the mutex or condition variable. The main thread detects the flag in its polling loop (50ms sleep intervals), then acquires `queue_mutex` normally and cancels all remaining orders. After cancellation, it calls `pthread_cond_broadcast` to wake all waiting couriers. Each courier checks the `shutdown_flag` after waking from `cond_wait` and exits cleanly. The `SA_RESTART` flag is intentionally not set so that blocked system calls (including `usleep` in the main loop) are interrupted by the signal, allowing faster shutdown detection.

Challenge 2: Ensuring Delivery Completion Is Not Interrupted by SIGINT

Problem: When a courier is in the middle of delivering (sleeping via `usleep`), SIGINT could potentially interrupt the `usleep` call, causing the delivery to complete prematurely with less than the full duration. The homework explicitly requires that active deliveries finish without interruption.

Solution: The delivery simulation uses a loop of small sleep chunks (100ms each) rather than a single large `usleep` call. Each chunk's remaining time is tracked, and if `usleep` is interrupted by a signal (returning early), the loop simply continues with the remaining time. The `shutdown_flag` is only checked at the top of the courier's main loop (when trying to dequeue), never during delivery. This means even if SIGINT arrives mid-delivery, the courier will complete the full delivery duration before checking whether it should exit. The delivery is never interrupted, and the atomic counter updates and `DELIVERY_COMPLETE` log are always printed.

Challenge 3: Preventing Duplicate Order Processing

Problem: With multiple courier threads competing for orders, there is a risk that two couriers could pop the same order if the synchronization is not correct. This would result in the same order being delivered twice, which is a critical correctness bug.

Solution: The `queue_mutex` ensures that only one courier can access the priority queue at a time. The critical section is: lock mutex, check if queue has items, pop the top item, unlock mutex. Since `heap_pop` atomically removes the item from the queue while holding the mutex, no other thread can see or pop the same item. The mutex is released only after the order has been removed, so there is zero window for duplicate processing. This was verified in Test Scenario #5 by checking for duplicate `DELIVERY_START` ids across multiple runs with 10 concurrent courier threads.